

Universidad Rafael Landívar

Facultad Ingeniería

Ingeniería en Sistemas e Informática

Ciencia de Datos

Inge. Max Cerna

Proyecto 01 – Fase 01

PROYECTO RED METROPOLITANA

Rochelle Giulianne Esquivel Vargas – 1283220

Susana Paola García García – 1224323

Harriett Alexandra Guzmán López – 1112423

Guatemala, -- de septiembre del 2026

Tabla de contenido

INTRODUCCIÓN	5
OBJETIVOS	6
OBJETIVO GENERAL	6
OBJETIVOS ESPECÍFICOS.....	6
METODOLOGÍA Y ARQUITECTURA DEL PROYECTO.....	9
MÉTODO GENERAL DE TRABAJO	9
ARQUITECTURA SELECCIONADA.....	9
SEPARACIÓN ENTRE LAS CAPAS.....	9
ORDEN DE DISEÑO	9
DECISIONES DE INGESTA Y ALMACENAMIENTO BRONZE.....	9
MÉTODOS DE INGESTA POR FUENTE.....	9
MÉTODO ELEGIDO PARA TRANSURBANO	10
UBICACIÓN DE LA CAPA BRONZE.....	10
FORMATO DE ALMACENAMIENTO	10
DECISIONES DE STAGING Y PROCESAMIENTO CDC	10
USO DE STAGING	10
APLICACIÓN DE LAS OPERACIONES CDC.....	10
RESULTADO DEL PADRÓN VIGENTE	11
CATÁLOGOS MÍNIMOS DE USUARIOS.....	11
DECISIONES DE INTEGRACIÓN Y CALIDAD EN SILVER.....	11
UNIFICACIÓN DE FORMATOS	11
CONFORMACIÓN DE ZONAS	11
ESTRATEGIA DE IDENTIDAD DEL USUARIO	11
TRATAMIENTO DE LOS REGISTROS INVÁLIDOS.....	11
HISTORIZACIÓN DEL PADRÓN	12
DECISIONES DEL MODELO DIMENSIONAL GOLD	12
GRANO DE LA TABLA DE HECHOS PRINCIPAL	12
SEPARACIÓN DE LAS TABLAS DE HECHOS	12
DIMENSIONES CONFORMADAS	12
CLASIFICACIÓN DE LAS MEDIDAS	12

ORQUESTACIÓN PRUEBAS MÉTRICAS E IDEMPOTENCIA	13
HERRAMIENTA DE TRANSFORMACIÓN	13
ORQUESTACIÓN DEL FLUJO.....	13
COMPROBACIÓN DE IDEMPOTENCIA.....	13
MÉTRICAS DOCUMENTADAS	13
GOBERNANZA SEGURIDAD Y ALCANCE POSTERIOR.....	13
DEFINICIONES OFICIALES	13
RESPONSABILIDAD Y LINAJE	13
DOCUMENTACIÓN Y REPRODUCIBILIDAD	14
SEGURIDAD Y DATOS PERSONALES.....	14
ALCANCE DE LA FASE 02.....	14
ANEXO.....	¡ERROR! MARCADOR NO DEFINIDO.
INFRAESTRUCTURA Y SERVICIOS	17
<i>Servicios de infraestructura en Docker Desktop</i>	<i>17</i>
<i>Verificación de servicios con Docker Compose</i>	<i>18</i>
INGESTA Y CAPA STAGING	19
<i>Conteo de registros por fuente en Staging</i>	<i>19</i>
PROCESAMIENTO CDC Y USUARIOS	20
<i>Operaciones CDC procesadas</i>	<i>20</i>
<i>Estado final del padrón de usuarios</i>	<i>21</i>
<i>Catálogos mínimos de usuarios por operador.....</i>	<i>22</i>
CALIDAD Y CONFORMACIÓN EN SILVER	23
<i>Validación de viajes de MetroRiel</i>	<i>23</i>
<i>Registros de cuarentena por tabla.....</i>	<i>24</i>
<i>Detalle de cuarentena por motivo de rechazo.....</i>	<i>25</i>
MODELO DIMENSIONAL GOLD	26
<i>Conteos del modelo dimensional Gold.....</i>	<i>26</i>
ORQUESTACIÓN E IDEMPOTENCIA	27
<i>Anexo de idempotencia del pipeline.....</i>	<i>27</i>
ALMACENAMIENTO Y PRUEBAS.....	28
<i>Tamaño de la capa Bronze.....</i>	<i>28</i>
<i>Resultado de las pruebas automatizadas de dbt</i>	<i>28</i>
<i>Tamaño de las capas almacenadas en PostgreSQL</i>	<i>29</i>
<i>Medición consolidada de todas las capas.....</i>	<i>30</i>

Introducción

Este documento reúne las Anexos técnicas obtenidas durante la implementación y validación de la Fase 01 del Proyecto Red Metropolitana de Transporte. Las capturas respaldan el funcionamiento de la infraestructura, la carga de datos, el procesamiento CDC, las reglas de calidad, el modelo dimensional y la auditoría del pipeline.

Cada Anexo se presenta con un título, una descripción interpretativa y la captura correspondiente. Los valores se explican en relación con su propósito dentro de la arquitectura, de modo que el documento pueda utilizarse como anexo de validación y como referencia para la revisión académica del proyecto.

La conclusión general es que el flujo funciona de extremo a extremo y conserva la trazabilidad de los datos. Las dos ejecuciones evaluadas produjeron los mismos conteos, las reglas de calidad separaron los registros inválidos y las pruebas de dbt finalizaron sin errores.

Objetivos

Objetivo general

Documentar las decisiones de arquitectura, los métodos aplicados y los resultados que demuestran el funcionamiento de la Fase 01 del pipeline de la Red Metropolitana.

Objetivos específicos

- Explicar la selección de los métodos de ingesta, almacenamiento y transformación.
- Describir el tratamiento del padrón CDC, la identidad y las reglas de calidad.
- Justificar el grano y la estructura del modelo dimensional Gold.
- Presentar las métricas, pruebas y Anexos de idempotencia del pipeline.

Problemática

El sistema de transporte del área metropolitana de Guatemala está conformado por distintos operadores que administran su información de manera independiente. Transmetro, Transurbano, MetroRiel y Aerometro utilizan formatos, estructuras e identificadores diferentes, lo cual dificulta integrar los datos y obtener una visión completa del funcionamiento de la red.

Las fuentes presentan diferencias importantes. Algunas registran abordajes individuales, mientras que MetroRiel almacena viajes completos con entrada y salida. También existen variaciones en los formatos de fecha, zonas, monedas, zonas horarias y llaves de usuario. Por ejemplo, Transurbano registra los montos en centavos, Aerometro utiliza horario UTC y cada operador identifica a sus usuarios de una manera distinta.

Además, el padrón de usuarios de Transmetro contiene operaciones CDC de inserción, actualización y eliminación que deben procesarse respetando su orden. Los registros eliminados no pueden borrarse físicamente porque esto provocaría la pérdida del historial de viajes asociado con cada tarjeta.

También se encontraron problemas de calidad, como validaciones duplicadas, llaves inexistentes, códigos de parada nulos, fechas futuras y viajes de MetroRiel sin estación de salida. Descartar estos datos impediría conocer el origen de los errores y reduciría la trazabilidad del sistema.

Por lo tanto, la problemática principal consiste en la ausencia de una plataforma unificada que permita integrar, limpiar, validar y analizar la información de los cuatro operadores sin perder los datos originales ni el historial de los usuarios.

Solución Implementada

Para resolver la problemática se implementó un pipeline de ingeniería de datos organizado mediante una arquitectura por capas. La solución integra los diferentes métodos de ingesta, conserva los datos originales, aplica reglas de calidad y construye un modelo dimensional preparado para el análisis.

La ingesta se dividió de acuerdo con las características de cada fuente. Las validaciones de Transmetro y los abordajes de Aerometro se procesaron mediante streaming con Kafka, debido a que representan eventos que podrían recibirse continuamente. Los catálogos, los viajes de MetroRiel y las transacciones de Transurbano se procesaron mediante cargas batch. El padrón de usuarios de Transmetro se trató como CDC porque contiene inserciones, actualizaciones y eliminaciones.

Los archivos recibidos se almacenaron en la capa Bronze mediante carpetas y archivos Parquet particionados por fecha. Esta capa conserva la información original y permite reprocesar las fuentes cuando sea necesario. Posteriormente, los datos se cargaron temporalmente en Staging dentro de PostgreSQL.

En Silver se unificaron los formatos de fecha, hora, moneda, zona e identificadores. También se procesó el padrón CDC, se construyeron los catálogos mínimos de usuarios y se aplicaron las reglas de calidad. Los registros inválidos no se eliminaron, sino que fueron trasladados a cuarentena junto con el motivo específico del rechazo.

La capa Gold se construyó exclusivamente a partir de Silver. Se crearon cinco dimensiones conformadas y dos tablas de hechos: una para los abordajes y otra para los viajes completos de MetroRiel. Las llaves originales de los usuarios fueron reemplazadas por identificadores seudonimizados para proteger la información personal.

Las transformaciones y pruebas se desarrollaron con dbt, mientras que Prefect se utilizó para ejecutar y controlar las etapas del pipeline. La infraestructura se levantó con Docker y los resultados de cada corrida se registraron en el esquema de auditoría.

Metodología y arquitectura del proyecto

Las siguientes decisiones responden, una por una, a los criterios indicados en el documento del proyecto. Se presentan en la misma secuencia lógica utilizada para construir y validar la solución.

Método general de trabajo

Se aplicó una metodología de ingeniería de datos por capas. Las fuentes ingresan mediante streaming, batch o CDC; se conservan en Bronze; se cargan temporalmente en Staging; se limpian y conforman en Silver; y finalmente se modelan en Gold para análisis. Cada etapa tiene una responsabilidad definida y puede validarse por separado.

Arquitectura seleccionada

La arquitectura combina Kafka en Docker para streaming, un lake local con archivos Parquet para Bronze, PostgreSQL para Staging, Silver, Gold, cuarentena y auditoría, dbt para las transformaciones y pruebas, y Prefect para la orquestación. Tableau queda establecido como consumidor de Gold para la Fase 02.

Separación entre las capas

Gold nunca consulta Bronze directamente. Los datos siempre pasan por Silver para recibir limpieza, normalización y controles de calidad. Los registros inválidos se almacenan en cuarentena, Bronze se conserva y Staging se vacía al finalizar cada ejecución.

Orden de diseño

El modelo se definió a partir de los procesos de negocio y de la matriz del bus. Este orden permitió identificar primero los granos, las tablas de hechos y las dimensiones conformadas antes de construir físicamente las tablas Gold.

Decisiones de ingesta y almacenamiento Bronze

Métodos de ingesta por fuente

Las validaciones de Transmetro y los abordajes de Aerometro se procesan mediante streaming con Kafka porque representan eventos continuos. Los catálogos y los viajes

completos de MetroRiel se cargan por batch. El padrón de usuarios utiliza CDC porque contiene inserciones, actualizaciones y eliminaciones.

Método elegido para Transurbano

Transurbano se procesa mediante batch porque la fuente fue entregada como un archivo completo de transacciones y no como un flujo continuo. Esta decisión simplifica la carga y la hace reproducible, aunque la información se actualiza por corrida y no en tiempo real.

Ubicación de la capa Bronze

Bronze se almacena en un lake local organizado en carpetas y particiones por fecha. Esta alternativa permite conservar los datos tal como llegaron, mantener trazabilidad y volver a procesarlos sin modificar la fuente original.

Formato de almacenamiento

Se seleccionó Parquet porque conserva tipos de datos, ofrece compresión y reduce el espacio utilizado. También permite guardar el contenido anidado de MetroRiel sin imponer inmediatamente una estructura relacional. La medición realizada reportó 67 MB en Bronze.

Decisiones de Staging y procesamiento CDC

Uso de Staging

Staging funciona como una zona temporal de carga. Recibe los archivos de cada ejecución y se vacía cuando finaliza el pipeline. Bronze permanece conservado, por lo que siempre existe una fuente desde la cual reprocesar los datos.

Aplicación de las operaciones CDC

El padrón se procesa respetando el orden de la columna de secuencia. INSERT crea una tarjeta, UPDATE modifica sus atributos y DELETE la marca como inactiva. Los borrados son lógicos para no perder el historial de viajes asociado con la tarjeta.

Resultado del padrón vigente

Se procesaron 31,050 operaciones CDC: 10,800 inserciones, 16,200 actualizaciones y 4,050 eliminaciones. El estado final contiene 17,432 tarjetas, de las cuales 15,096 están activas y 2,336 inactivas.

Catálogos mínimos de usuarios

Aerometro, MetroRiel y Transurbano no entregaron un padrón. Para cada operador se construyó un catálogo con las llaves distintas encontradas en sus operaciones, sin inventar nombres ni otros atributos. Se identificaron 14,496 usuarios de Aerometro, 22,885 de MetroRiel y 36,567 de Transurbano.

Decisiones de integración y calidad en Silver

Unificación de formatos

En Silver se estandarizaron fechas y horas, se convirtió la hora UTC de Aerometro a la hora local de Guatemala y se transformaron los centavos de Transurbano a quetzales. También se limpiaron espacios, identificadores y nombres de operadores.

Conformación de zonas

Las distintas representaciones de una zona se normalizaron para apuntar a una misma llave. Por ejemplo, Z10, Zona 10 y su equivalencia proveniente del campo district se interpretan como la misma zona dentro del modelo integrado.

Estrategia de identidad del usuario

Las llaves de origen se normalizan y se relacionan con su operador. Antes de Gold se genera una llave sustituta seudonimizada mediante un hash determinístico. La limitación es que no existe una tabla oficial que demuestre que tarjetas diferentes pertenecen a la misma persona; por ello, la identificación multimodal es una aproximación declarada.

Tratamiento de los registros inválidos

Los registros que incumplen una regla no se descartan. Se trasladan a cuarentena junto con el motivo de rechazo. Se controlan llaves ausentes o no reconocidas, viajes sin salida,

validaciones duplicadas, códigos de parada nulos y fechas futuras. El total documentado es de 18,431 registros.

Historización del padrón

El padrón se historiza con el enfoque SCD Tipo 2. Los cambios generan versiones con un periodo de vigencia y un indicador de registro actual, de manera que sea posible consultar tanto el estado presente como la evolución de los atributos del usuario.

Decisiones del modelo dimensional Gold

Grano de la tabla de hechos principal

El grano de fct_abordajes es un abordaje válido realizado por un usuario, en un modo de transporte, ubicación y fecha y hora determinadas. Cada fila representa un evento individual y no un resumen diario o mensual.

Separación de las tablas de hechos

Se crearon fct_abordajes y fct_viajes_metroriel porque no representan exactamente el mismo proceso. Los primeros tres operadores registran abordajes, mientras que MetroRiel entrega un viaje completo con entrada, salida y duración. Mantener dos hechos evita perder ese detalle.

Dimensiones conformadas

Las tablas de hechos comparten las dimensiones dim_fecha, dim_hora, dim_modos, dim_usuario y dim_zona. Estas dimensiones mantienen definiciones y llaves comunes para comparar operadores sin repetir reglas de interpretación.

Clasificación de las medidas

El monto, la cantidad de abordajes y la cantidad de viajes son medidas aditivas. Los estados o saldos observados a través del tiempo son semi aditivos. Los porcentajes, promedios, duraciones promedio y usuarios distintos son no aditivos y deben recalcularse para cada nivel de análisis.

Orquestación pruebas métricas e idempotencia

Herramienta de transformación

Se eligió dbt porque permite organizar los modelos SQL, declarar dependencias, ejecutar pruebas y generar linaje. La validación final registró 79 pruebas aprobadas, sin errores, advertencias ni pruebas omitidas.

Orquestación del flujo

Prefect coordina la ingesta hacia Bronze, la carga de Staging, la ejecución de dbt build, el registro de auditoría y la limpieza de Staging. También registra el resultado y la duración de las etapas para facilitar el seguimiento del pipeline.

Comprobación de idempotencia

El pipeline se ejecutó dos veces y conservó los mismos conteos: 1,730,184 registros en Staging, 1,919,887 en Silver, 1,718,037 en Gold y 18,431 en cuarentena. La igualdad demuestra que una nueva corrida no introduce duplicados.

Métricas documentadas

Se reportaron filas por fuente, conteos de Silver y Gold, registros de cuarentena por motivo, operaciones CDC, usuarios activos e inactivos, duración de las corridas y tamaño en disco por capa. Estas cifras convierten las afirmaciones del proyecto en resultados medidos.

Gobernanza seguridad y alcance posterior

Definiciones oficiales

Se define un abordaje como un acceso válido registrado por un operador; un viaje de MetroRiel como un trayecto con entrada y salida; un usuario activo como una tarjeta cuyo último estado CDC es activo; y un registro inválido como aquel que se conserva en cuarentena por incumplir una regla.

Responsabilidad y linaje

El equipo de datos administra las dimensiones de fecha y hora; el área de planificación autoriza cambios en la dimensión zona; operaciones validas los modos de transporte; y

gobernanza y seguridad administran la dimensión usuario. El linaje de dbt permite rastrear los modelos hasta las fuentes originales.

Documentación y reproducibilidad

El repositorio contiene requisitos, pasos de instalación, instrucciones para ejecutar el flujo y las decisiones de diseño. El historial de Git conserva los cambios realizados durante el desarrollo y no se limita a una única carga final.

Seguridad y datos personales

Las credenciales se administran con variables de entorno y el repositorio incluye únicamente un archivo de ejemplo. Las llaves se seudonimizan antes de Gold. Los analistas consultan información agregada, mientras que el detalle individual queda restringido a personal autorizado. Se propone conservar el detalle durante 12 meses y después mantener datos agregados o anonimizados.

Alcance de la Fase 02

La conexión de Tableau, el análisis de demanda y cobertura, los transbordos, la evaluación del trazado de MetroRiel, la recomendación de movilidad y la tabla de features corresponden a la Fase 02. La Fase 01 deja preparados Gold, Silver y el linaje necesario para esos productos.

Conclusión

- La arquitectura implementada permitió integrar correctamente fuentes con estructuras, formatos y niveles de detalle diferentes. La separación en Bronze, Staging, Silver y Gold facilitó controlar el procesamiento y mantener la trazabilidad de los datos.
- Los métodos de ingesta fueron seleccionados según las características de cada fuente. El uso de streaming, batch y CDC permitió representar adecuadamente eventos continuos, archivos periódicos y cambios en el padrón de usuarios.
- La capa Bronze conservó los datos originales en archivos Parquet, lo cual permite realizar nuevos procesamientos sin depender de las tablas transformadas. El tamaño medido de esta capa fue de 67 MB.
- El procesamiento CDC permitió aplicar 31,050 operaciones y construir el estado vigente del padrón. Como resultado, se obtuvieron 15,096 tarjetas activas y 2,336 tarjetas inactivas.
- Las reglas de calidad identificaron 18,431 registros inválidos. Estos datos fueron almacenados en cuarentena con su respectivo motivo, en lugar de ser eliminados silenciosamente.
- El modelo dimensional Gold quedó conformado por cinco dimensiones y dos tablas de hechos. En esta capa se almacenaron 1,718,037 registros preparados para el análisis posterior en Tableau.
- Las 79 pruebas configuradas en dbt finalizaron correctamente, sin errores, advertencias ni pruebas omitidas. Esto respalda la integridad y consistencia de los modelos construidos.
- La idempotencia del pipeline fue comprobada mediante dos ejecuciones consecutivas. Ambas conservaron los mismos conteos en Staging, Silver, Gold y cuarentena, demostrando que una nueva corrida no genera duplicados.
- La Fase 01 proporciona una base técnica completa y reproducible para continuar con la visualización, el análisis de cobertura, la evaluación de la demanda y la construcción de features durante la Fase 02.

Recomendaciones

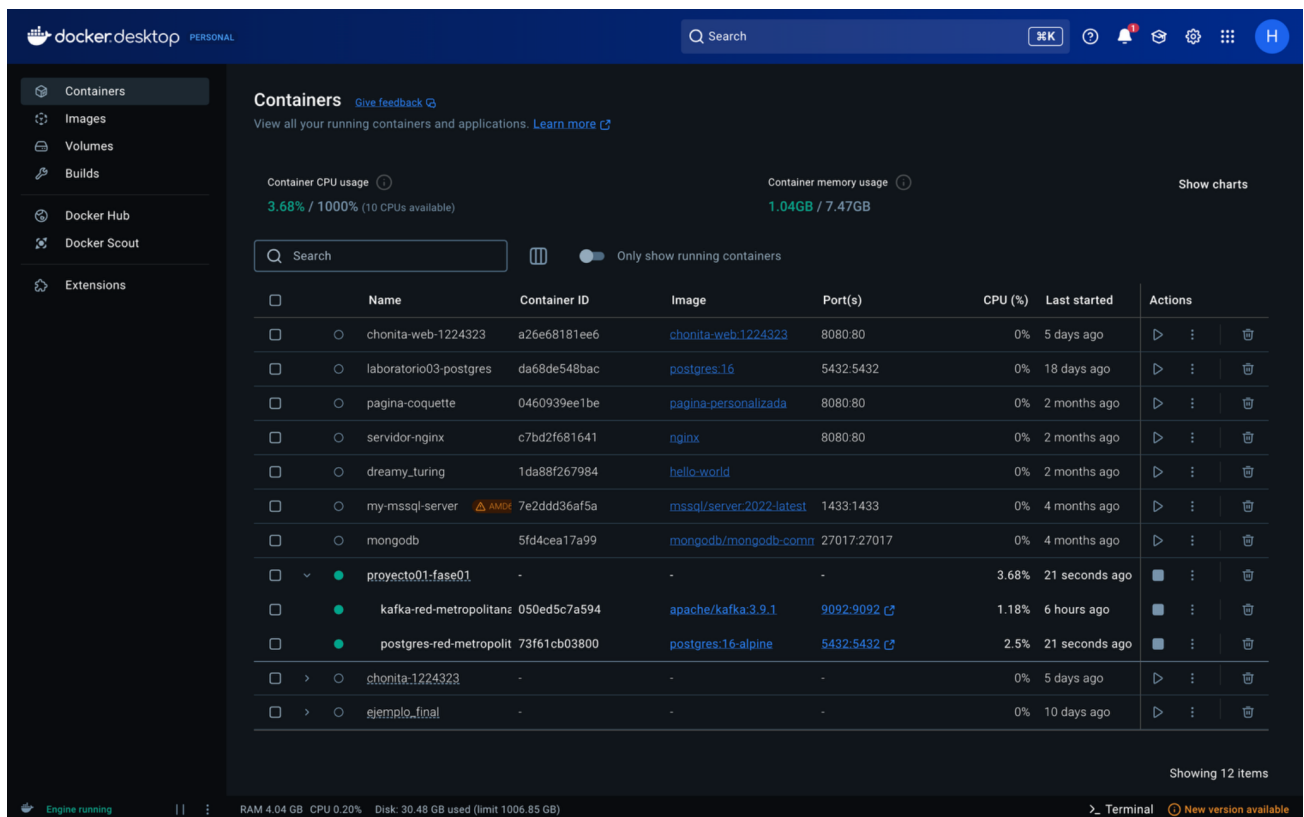
- Implementar el tablero de la Fase 02 conectándolo exclusivamente con la capa Gold, evitando consultar directamente las fuentes originales, Bronze o Staging.
- Incorporar alertas de frescura que permitan detectar cuando una fuente no haya llegado dentro del periodo esperado o presente una reducción anormal en la cantidad de registros.
- Configurar GitHub Actions para ejecutar automáticamente las pruebas de dbt cada vez que se realice un cambio en el repositorio. Esto permitiría detectar errores antes de integrar nuevas modificaciones.
- Evaluar el uso de modelos incrementales de dbt cuando aumente el volumen de información, de manera que solamente se procesen los registros nuevos o modificados.
- Fortalecer la estrategia de identidad mediante una tabla oficial que relacione las tarjetas de los diferentes operadores. Esto permitiría identificar con mayor precisión a los usuarios multimodales.
- Mantener documentadas las definiciones oficiales de abordaje, viaje, usuario activo, zona y registro inválido para garantizar que todos los integrantes calculen las métricas de la misma manera.
- Asignar responsables para las dimensiones conformadas. Los cambios en zonas, modos, usuarios o reglas de calidad deben ser revisados y autorizados antes de aplicarse en producción.
- Restringir el acceso al detalle individual de los viajes. Los analistas deberían consultar información agregada, mientras que los datos personales solamente deben estar disponibles para usuarios autorizados.
- Definir una política de retención. Se recomienda conservar el detalle individual durante 12 meses y, después de ese periodo, mantener únicamente información agregada o anonimizada.
- Conservar las evidencias de ejecución, los conteos y las métricas de rendimiento para comparar futuras versiones del pipeline y detectar cambios inesperados.

Anexos

Infraestructura y Servicios

Servicios de infraestructura en Docker Desktop

La captura muestra el grupo de contenedores del proyecto en Docker Desktop. Kafka y PostgreSQL se encuentran en ejecución y exponen los puertos 9092 y 5432, respectivamente. Esta Anexo confirma la disponibilidad de la infraestructura requerida para la mensajería y el almacenamiento relacional del pipeline.



☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	chonita-web-1224323	a26e68181ee6	chonita-web:1224323	8080:80	0%	5 days ago	▶ ⋮ 🗑
☐	laboratorio03-postgres	da68de548bac	postgres:16	5432:5432	0%	18 days ago	▶ ⋮ 🗑
☐	pagina-coquette	0460939ee1be	pagina-personalizada	8080:80	0%	2 months ago	▶ ⋮ 🗑
☐	servidor-nginx	c7bd2f681641	nginx	8080:80	0%	2 months ago	▶ ⋮ 🗑
☐	dreamy_turing	1da88f267984	hello-world		0%	2 months ago	▶ ⋮ 🗑
☐	my-mssql-server	7e2ddd36af5a	mssql/server:2022-latest	1433:1433	0%	4 months ago	▶ ⋮ 🗑
☐	mongodb	5fd4cea17a99	mongodb/mongodb-community	27017:27017	0%	4 months ago	▶ ⋮ 🗑
☐	proyecto01-fase01	-	-	-	3.68%	21 seconds ago	▶ ⋮ 🗑
☐	kafka-red-metropolitana	050ed5c7a594	apache/kafka:3.9.1	9092:9092	1.18%	6 hours ago	▶ ⋮ 🗑
☐	postgres-red-metropolitano	73f61cb03800	postgres:16-alpine	5432:5432	2.5%	21 seconds ago	▶ ⋮ 🗑
☐	chonita:1224323	-	-	-	0%	5 days ago	▶ ⋮ 🗑
☐	ejemplo_final	-	-	-	0%	10 days ago	▶ ⋮ 🗑

Showing 12 items

Engine running | RAM 4.04 GB | CPU 0.20% | Disk 30.48 GB used (limit 1006.85 GB) | Terminal | New version available

Anexo 1. Contenedores Kafka y PostgreSQL activos en Docker Desktop.

Verificación de servicios con Docker Compose

Se ejecutaron los comandos de Docker Compose para iniciar los servicios y consultar su estado. La salida confirma que Kafka y PostgreSQL fueron creados y levantados con sus puertos configurados, lo que permite reproducir el entorno de trabajo de la Fase 01.

The screenshot shows a VS Code editor with a project named 'Proyecto 01 - Fase 01'. The file explorer on the left shows a directory structure including .venv, .venv_python314_backup, datos_red, dbt_red_metropolitana, docs, lake, logs, orchestration, scripts, .env, .env.example, .gitignore, docker-compose.yml, generar_red_metropolitana.py, README.md, and requirements.txt. The terminal window at the bottom shows the execution of 'docker compose up -d' and 'docker compose ps'. The output of 'docker compose ps' is as follows:

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
kafka-red-metropolitana	apache/kafka:3.9.1	"/__cncert_entrypoint..."	kafka	2 days ago	Up 6 hours	0.0.0.0:9092->9092/tcp
postgres-red-metropolitana	postgres:16-alpine	"docker-entrypoint.s..."	postgres	9 seconds ago	Up 8 seconds (health: starting)	0.0.0.0:5432->5432/tcp

The database explorer on the right shows a connection to 'postgres@Red Metropolitana' with 6 of 8 server objects and 3 query files.

Anexo 2. Inicio y comprobación de los servicios mediante Docker Compose.

Ingesta y Capa Staging

Conteo de registros por fuente en Staging

La consulta valida la carga de las nueve fuentes en el esquema Staging. En conjunto se registran 1,730,184 filas: transacciones, validaciones, viajes, abordajes, catálogos de estaciones y paradas, además del padrón CDC. El total coincide con el volumen de entrada documentado para la fase.

The screenshot shows a database console interface with a SQL query and its results. The query is a UNION ALL statement that counts records from various staging tables. The results are displayed in a table with two columns: 'tabla' and 'registros'.

```
57 UNION ALL
58
59 SELECT tabla 'metroriel_viajes_raw', registros COUNT(*)
60 FROM staging.metroriel_viajes_raw
61
62 UNION ALL
63
64 SELECT tabla 'aerometro_boardings', registros COUNT(*)
65 FROM staging.aerometro_boardings
66
67 UNION ALL
68
69 SELECT tabla 'cdc_padron_usuarios', registros COUNT(*)
70 FROM staging.cdc_padron_usuarios
```

tabla	registros
1 aerometro_boardings	203554
2 am_estaciones	14
3 cdc_padron_usuarios	31050
4 metroriel_viajes_raw	299100
5 mr_estaciones	22
6 tm_estaciones	104
7 transmetro_validaciones	363221
8 transurbano_transacciones	832791
9 tv_paradas	328

Anexo 3. Distribución de los 1,730,184 registros cargados en Staging.

Procesamiento CDC y Usuarios

Operaciones CDC procesadas

El archivo del padrón contiene 31,050 operaciones de cambio: 10,800 inserciones, 16,200 actualizaciones y 4,050 eliminaciones lógicas. El desglose demuestra que las operaciones CDC fueron identificadas y contabilizadas antes de construir el estado actual de los usuarios.

The screenshot shows a database IDE with a project named 'Proyecto 01 - Fase 01'. The left sidebar displays the project structure, including files like 'ddl_gold.sql', 'diagrama_modelo.md', 'datos_red', 'dbt_red_metropolitana', 'docs', 'lake', 'logs', 'orchestration', 'scripts', and environment files like '.env' and '.env.example'. The main editor shows a SQL query in a console window titled 'validaciones [postgres@Red...etropolitana]'. The query is as follows:

```
132
133
134 -- 9. VALIDACIÓN DE CONEXIÓN A POSTGRESQL
135 SELECT
136     current_user AS usuario_conectado,
137     current_database() AS base_de_datos;
138
139 -- 10. OPERACIONES CDC PROCESADAS
140 SELECT 354 ms
141     op AS operacion,
142     COUNT(*) AS cantidad
143 FROM staging.cdc_padrón_usuarios
144 GROUP BY op
145 ORDER BY op;
```

The right sidebar shows the database structure, including 'postgres@Red Metropolitana' and 'red_metropolitana'. The bottom panel displays the 'Output' window with 'Result 10' showing the results of the query:

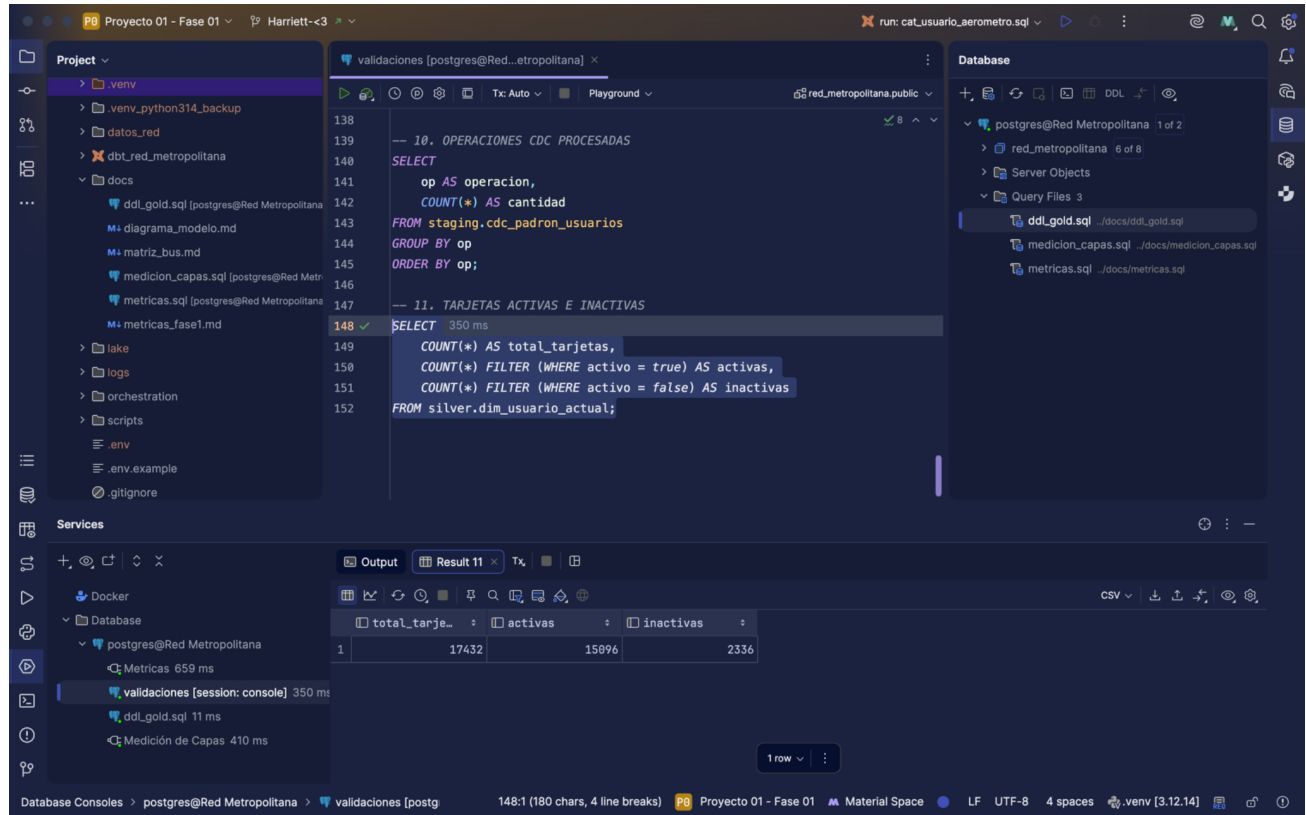
operacion	cantidad
1 DELETE	4050
2 INSERT	10800
3 UPDATE	16200

The bottom status bar indicates the current session is 'postgres@Red Metropolitana' and the query is 'validaciones [postgres@Red...etropolitana]' with 140:1 (110 chars, 5 line breaks). The IDE also shows the project name 'Proyecto 01 - Fase 01' and the file encoding 'UTF-8'.

Anexo 4. Conteo de operaciones INSERT, UPDATE y DELETE del padrón CDC.

Estado final del padrón de usuarios

Después de aplicar las operaciones CDC, el padrón conserva 17,432 tarjetas: 15,096 activas y 2,336 inactivas. Las eliminaciones se representan mediante un estado inactivo en lugar de borrar físicamente el registro, preservando así la trazabilidad histórica.



The screenshot shows a database console interface with a project explorer on the left, a SQL editor in the center, and a results pane at the bottom. The SQL editor contains two queries. The first query, labeled '10. OPERACIONES CDC PROCESADAS', is a SELECT statement that counts the number of operations by type. The second query, labeled '11. TARJETAS ACTIVAS E INACTIVAS', is a SELECT statement that counts the total number of cards, as well as the number of active and inactive cards. The results pane shows the output of the second query, which is a table with four columns: total_tarjetas, activas, and inactivas. The table contains one row of data: 17432 total cards, 15096 active cards, and 2336 inactive cards.

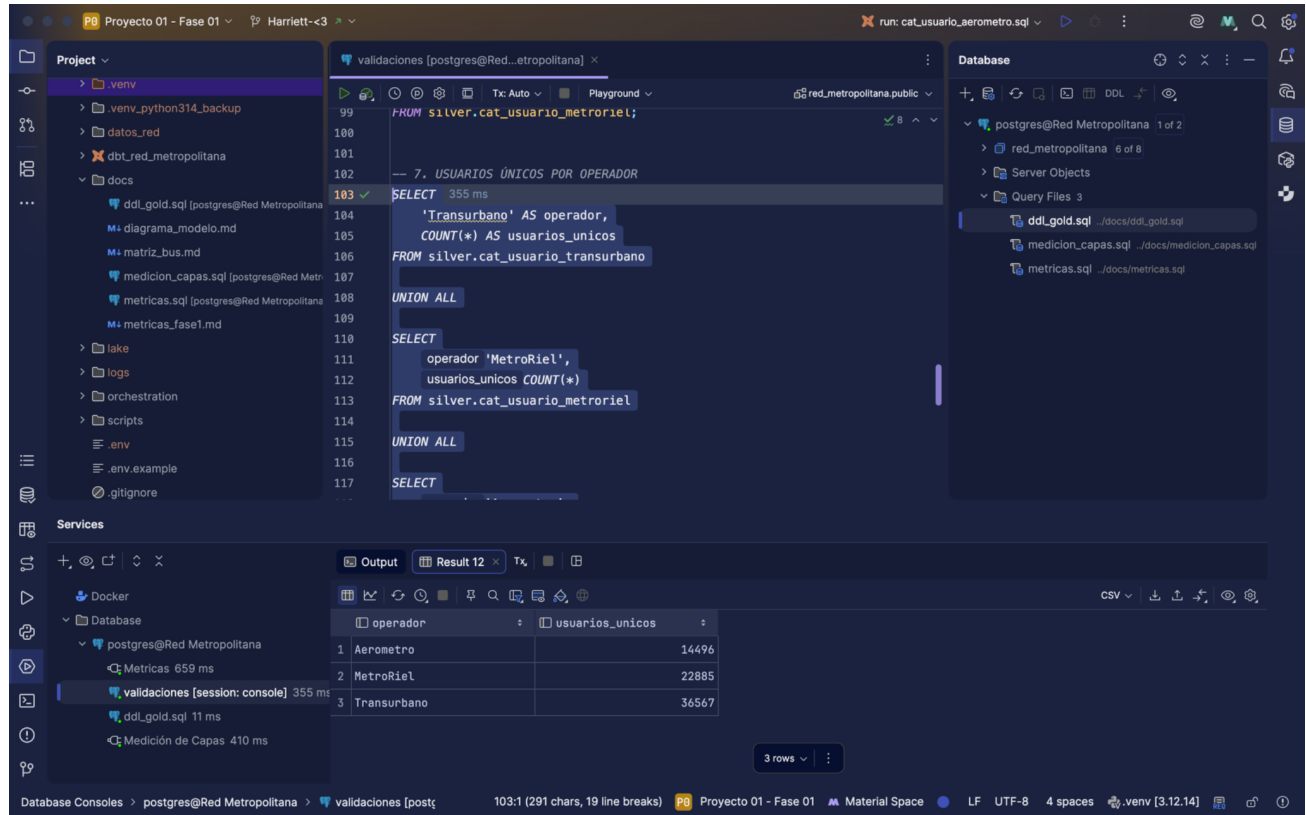
```
138 -- 10. OPERACIONES CDC PROCESADAS
139 SELECT
140     op AS operacion,
141     COUNT(*) AS cantidad
142 FROM staging.cdc_padrón_usuarios
143 GROUP BY op
144 ORDER BY op;
145
146 -- 11. TARJETAS ACTIVAS E INACTIVAS
147 SELECT
148     COUNT(*) AS total_tarjetas,
149     COUNT(*) FILTER (WHERE activo = true) AS activas,
150     COUNT(*) FILTER (WHERE activo = false) AS inactivas
151 FROM silver.dim_usuario_actual;
```

	total_tarjetas	activas	inactivas
1	17432	15096	2336

Anexo 5. Cantidad de tarjetas activas e inactivas en Silver.

Catálogos mínimos de usuarios por operador

Los catálogos mínimos reúnen las llaves de usuario distintas detectadas por operador: 14,496 para Aerometro, 22,885 para MetroRiel y 36,567 para Transurbano. Estos conteos sirven como base para conformar identidades entre las distintas fuentes del sistema.



The screenshot shows a database IDE interface with a project explorer on the left, a central SQL editor, and a results pane at the bottom. The SQL editor contains a query that counts unique users for three operators: Aerometro, MetroRiel, and Transurbano. The results pane displays the output of the query, showing 3 rows of data.

```
run: cat_usuario_aerometro.sql
Project: Proyecto 01 - Fase 01
validaciones [postgres@Red...etropolitana]
-- 7. USUARIOS ÚNICOS POR OPERADOR
SELECT
  'Transurbano' AS operador,
  COUNT(*) AS usuarios_unicos
FROM silver.cat_usuario_transurbano
UNION ALL
SELECT
  operador 'MetroRiel',
  usuarios_unicos COUNT(*)
FROM silver.cat_usuario_metroriel
UNION ALL
SELECT
```

operador	usuarios_unicos
Aerometro	14496
MetroRiel	22885
Transurbano	36567

Anexo 6. Usuarios distintos identificados en cada operador.

Calidad y Conformación en Silver

Validación de viajes de MetroRiel

De los 299,100 viajes originales de MetroRiel, 295,511 cumplen las reglas de calidad y pasan a Silver. Los 3,589 registros restantes no poseen salida y se trasladan a cuarentena. La suma de ambos resultados coincide con el total de origen.

The screenshot shows a database console interface with the following components:

- Project Explorer:** Lists files like `ddl_gold.sql`, `diagrama_modelo.md`, `matriz_bus.md`, `medicion_capas.sql`, `metricas.sql`, and `metricas_fase1.md`.
- Query Editor:** Displays SQL queries for validation. Query 4 calculates the final state of the CDC DE TRANSMETRO by comparing original, silver, and quarantined records. Query 5 calculates the total number of active tickets.
- Database Explorer:** Shows the database structure, including tables like `red_metropolitana`, `ddl_gold.sql`, `medicion_capas.sql`, and `metricas.sql`.
- Output/Results:** Shows the results of the queries. The first query returns a single row with counts for `registros_originales` (299100), `registros_silver` (295511), and `viajes_sin_salida` (3589).

```
-- 4. ESTADO FINAL DEL PADRÓN CDC DE TRANSMETRO
SELECT
  (SELECT COUNT(*)
   FROM staging.metroriel_viajes_raw)
   AS registros_originales,

  (SELECT COUNT(*)
   FROM silver.metroriel_viajes)
   AS registros_silver,

  (SELECT COUNT(*)
   FROM quarantine.metroriel_sin_salida)
   AS viajes_sin_salida;

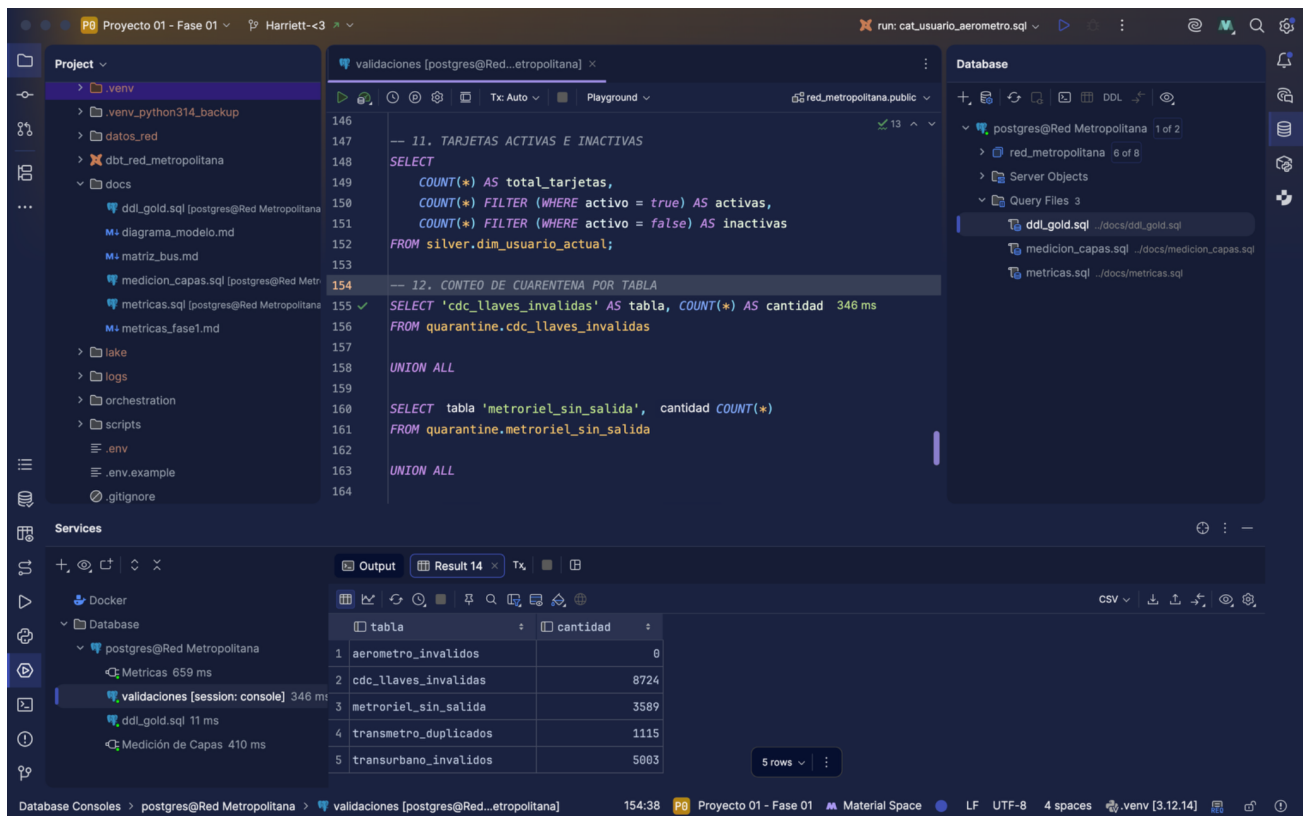
--5. ESTADO FINAL DEL PADRÓN CDC DE TRANSMETRO
SELECT
  COUNT(*) AS total_tarjetas,
  COUNT(*) FILTER (WHERE activa = true) AS tarjetas_activas;
```

	registros_originales	registros_silver	viajes_sin_salida
1	299100	295511	3589

Anexo 7. Conciliación entre viajes originales, válidos y enviados a cuarentena.

Registros de cuarentena por tabla

La consulta consolida 18,431 registros rechazados por las reglas de calidad. La mayor cantidad corresponde a llaves CDC inválidas, seguida por datos inválidos de Transurbano, viajes de MetroRiel sin salida y validaciones duplicadas de Transmetro. Aerometro no presenta registros inválidos.



The screenshot displays a database IDE interface with a project named 'Proyecto 01 - Fase 01'. The central pane shows a SQL query titled 'validaciones [postgres@Red...etropolitana]'. The query is divided into two parts: Part 11, 'TARJETAS ACTIVAS E INACTIVAS', and Part 12, 'CONTEO DE CUARENTENA POR TABLA'. Part 12 includes a SELECT statement for 'cdc_llaves_invalidas' and a UNION ALL with a SELECT statement for 'metroriel_sin_salida'. The results pane at the bottom shows a table with 5 rows and 2 columns: 'tabla' and 'cantidad'.

tabla	cantidad
aerometro_invalidos	0
cdc_llaves_invalidas	8724
metroriel_sin_salida	3589
transmetro_duplicados	1115
transurbano_invalidos	5003

Anexo 8. Conteo total de registros almacenados en cada tabla de cuarentena.

Detalle de cuarentena por motivo de rechazo

El detalle identifica el motivo específico de cada rechazo: llaves ausentes o ajenas al padrón, viajes sin salida, validaciones duplicadas, códigos de parada nulos y fechas futuras. Esta clasificación permite auditar la calidad, explicar las exclusiones y definir acciones de corrección por fuente.

The screenshot shows a database IDE with a project named 'Proyecto 01 - Fase 01'. The central pane displays a SQL query for 'validaciones [postgres@Red...etropolitana]'. The query is a UNION ALL of two SELECT statements, both counting records by rejection reason ('motivo_rechazo'). The first SELECT counts records from 'quarantine.transurbano_invalidos', and the second counts records from 'quarantine.aerometro_invalidos'. The results are ordered by the rejection reason.

The 'Database' pane on the right shows the schema for 'postgres@Red Metropolitana', including tables like 'ddl_gold.sql', 'medicion_capas.sql', and 'metricas.sql'.

The 'Output' pane at the bottom shows the results of the query, which are 7 rows. The columns are 'tabla', 'motivo', and 'cantidad'.

tabla	motivo	cantidad
cdc_llaves_invalidas	LLAVE_AUSENTE	2286
cdc_llaves_invalidas	LLAVE_NO_PERTENECE_AL_PADRON_TRANSMETRO	6518
metroriel_sin_salida	VIAJE_SIN_SALIDA	3589
transmetro_duplicados	VALIDACION_DUPLICADA	1115
transurbano_invalidos	CODIGO_PARADA_NULO	4186
transurbano_invalidos	CODIGO_PARADA_NULO FECHA_FUTURA	3
transurbano_invalidos	FECHA_FUTURA	814

Anexo 9. Distribución de registros de cuarentena según la regla incumplida.

Modelo Dimensional Gold

Conteos del modelo dimensional Gold

La capa Gold contiene cinco dimensiones conformadas y dos tablas de hechos. Destacan 1,352,058 abordajes, 295,511 viajes de MetroRiel y 70,380 usuarios conformados. El total de 1,718,037 registros confirman la construcción del modelo dimensional destinado al análisis.

The screenshot shows a database IDE interface with a project explorer on the left, a central SQL editor, and a results pane at the bottom. The project explorer lists files like `ddl_gold.sql`, `diagrama_modelo.md`, `matriz_bus.md`, `medicion_capas.sql`, `metricas.sql`, and `metricas_fase1.md`. The SQL editor displays a query titled "13. CONTEOS DEL MODELO DIMENSIONAL GOLD" which counts records for various dimensions and fact tables. The results pane shows a table with 7 rows and 2 columns: 'tabla' and 'cantidad'.

```
227 -- 13. CONTEOS DEL MODELO DIMENSIONAL GOLD
228 SELECT
229     'fct_abordajes' AS tabla,
230     COUNT(*) AS cantidad
231 FROM gold.fct_abordajes
232
233 UNION ALL
234
235 SELECT
236     'fct_viajes_metroriel' AS tabla,
237     COUNT(*) AS cantidad
238 FROM gold.fct_viajes_metroriel
239
240 UNION ALL
241
242 SELECT
243     'dim_usuario' AS tabla,
```

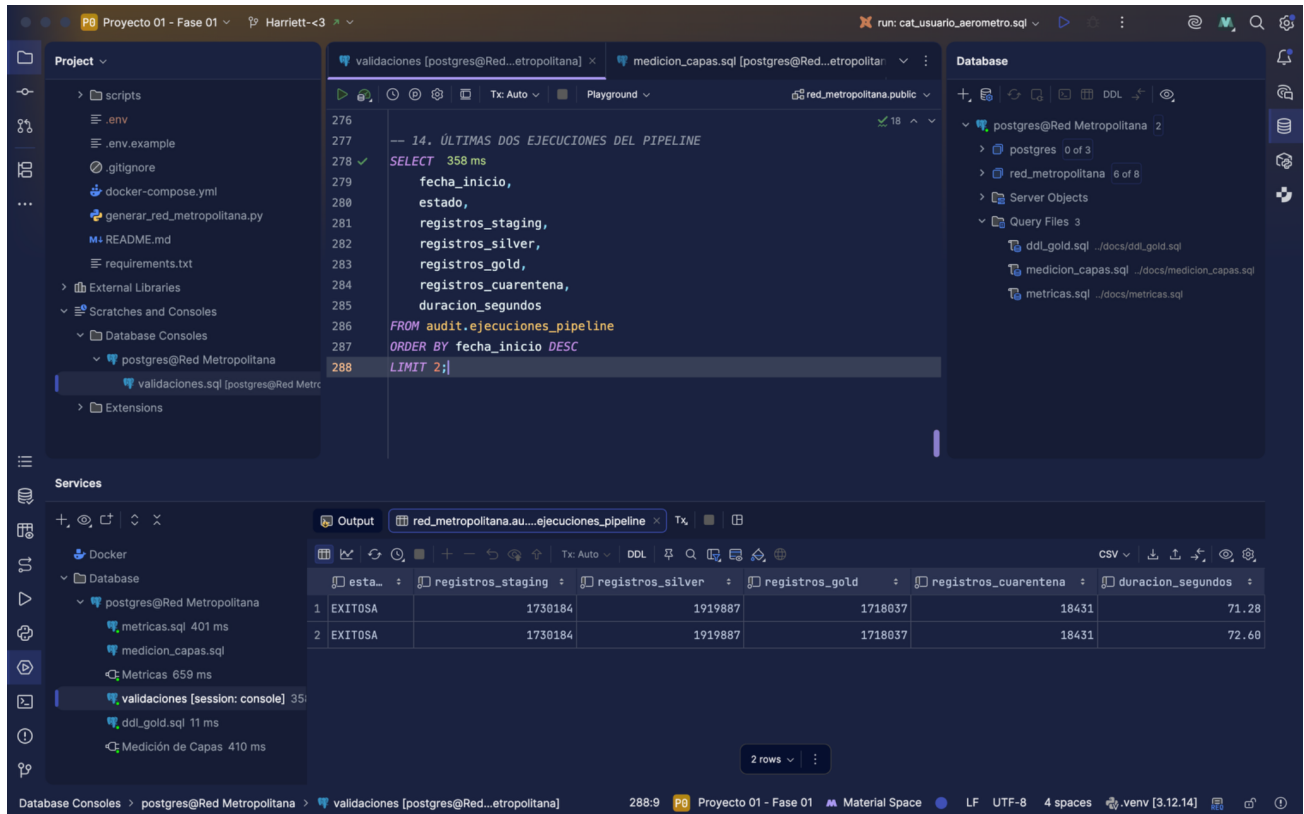
tabla	cantidad
dim_fecha	45
dim_hora	24
dim_mod0	4
dim_usuario	70380
dim_zona	15
fct_abordajes	1352058
fct_viajes_metroriel	295511

Anexo 10. Cantidad de registros en dimensiones y hechos del esquema Gold.

Orquestación e Idempotencia

Anexo de idempotencia del pipeline

Las dos ejecuciones más recientes finalizaron con estado EXITOSA y conservaron los mismos conteos: 1,730,184 registros en Staging, 1,919,887 en Silver, 1,718,037 en Gold y 18,431 en cuarentena. Las duraciones fueron 71.28 y 72.60 segundos. La igualdad de resultados demuestra que una nueva corrida no introduce duplicados.



The screenshot shows a database console interface with a project sidebar on the left, a central SQL editor, and a bottom output panel. The SQL editor contains a query to select the last two pipeline executions. The output panel displays a table with two rows of execution data.

```
-- 14. ÚLTIMAS DOS EJECUCIONES DEL PIPELINE
SELECT
  fecha_inicio,
  estado,
  registros_staging,
  registros_silver,
  registros_gold,
  registros_cuarentena,
  duracion_segundos
FROM audit.ejecuciones_pipeline
ORDER BY fecha_inicio DESC
LIMIT 2;
```

esta...	registros_staging	registros_silver	registros_gold	registros_cuarentena	duracion_segundos
1 EXITOSA	1730184	1919887	1718037	18431	71.28
2 EXITOSA	1730184	1919887	1718037	18431	72.60

Anexo 11. Dos ejecuciones exitosas con conteos idénticos.

Almacenamiento y Pruebas

Tamaño de la capa Bronze

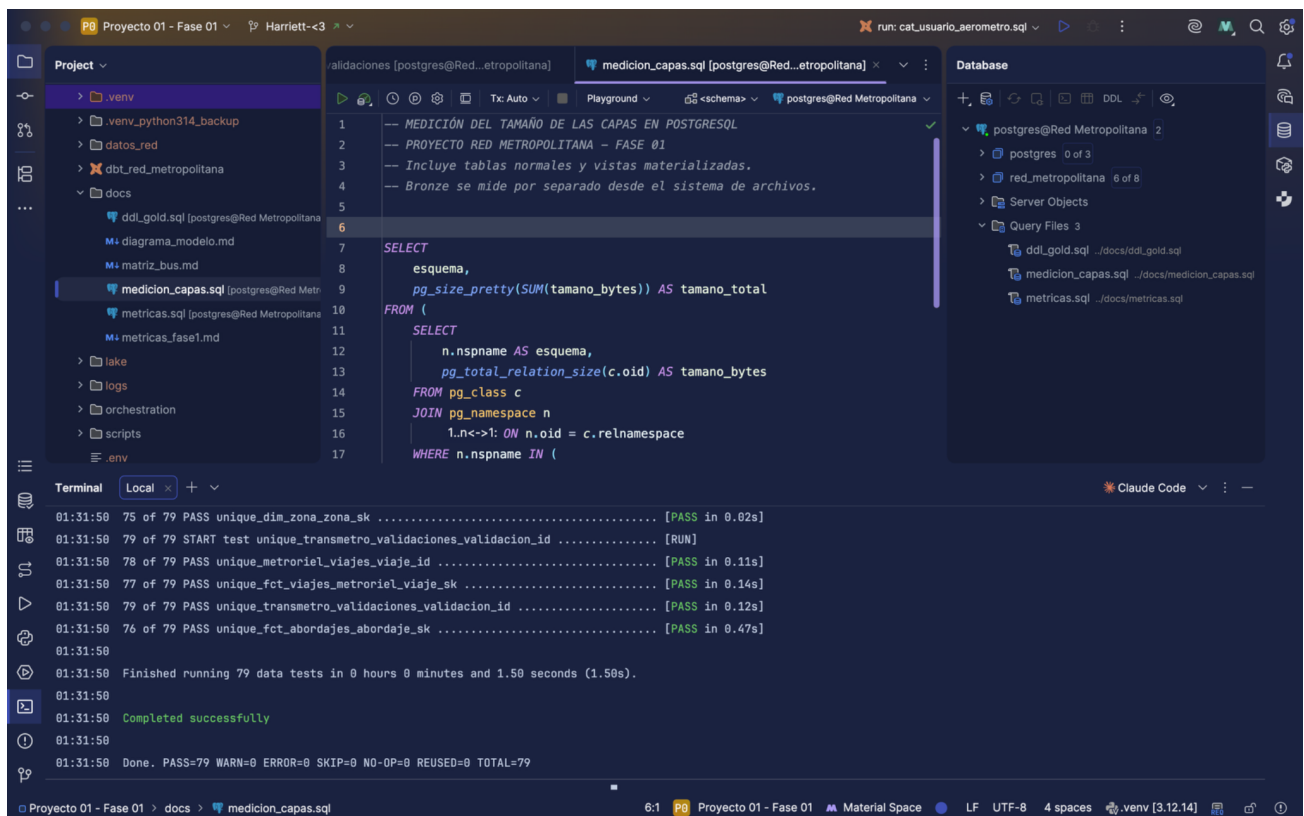
La capa Bronze se conserva fuera de PostgreSQL, en el sistema de archivos. El comando de medición reporta 67 MB para la carpeta lake/bronze, que contiene los archivos crudos utilizados para trazabilidad y reprocesamiento.

```
((.venv) ) (base) harriettguzman@MacBook-Pro-de-Harriett-3 Proyecto 01 - Fase 01 % du -sh lake/bronze
67M    lake/bronze
```

Anexo 12. Medición de 67 MB para la capa Bronze.

Resultado de las pruebas automatizadas de dbt

La ejecución de dbt finalizó correctamente: 79 pruebas aprobadas, sin advertencias, errores ni pruebas omitidas. El resultado respalda la validez de las reglas de unicidad, integridad y calidad definidas para los modelos de la Fase 01.



The screenshot shows a VS Code editor with a project named 'Proyecto 01 - Fase 01'. The file explorer on the left shows a directory structure with files like 'ddl_gold.sql', 'diagrama_modelo.md', 'medicion_capas.sql', and 'metricas.sql'. The main editor displays a SQL query in 'medicion_capas.sql' that calculates the size of tables in PostgreSQL. The query is as follows:

```
1 -- MEDICIÓN DEL TAMAÑO DE LAS CAPAS EN POSTGRESQL
2 -- PROYECTO RED METROPOLITANA - FASE 01
3 -- Incluye tablas normales y vistas materializadas.
4 -- Bronze se mide por separado desde el sistema de archivos.
5
6
7 SELECT
8     esquema,
9     pg_size_pretty(SUM(tamano_bytes)) AS tamano_total
10 FROM (
11     SELECT
12         n.nspname AS esquema,
13         pg_total_relation_size(c.oid) AS tamano_bytes
14     FROM pg_class c
15     JOIN pg_namespace n
16         ON n.oid = c.relnamespace
17     WHERE n.nspname IN (
```

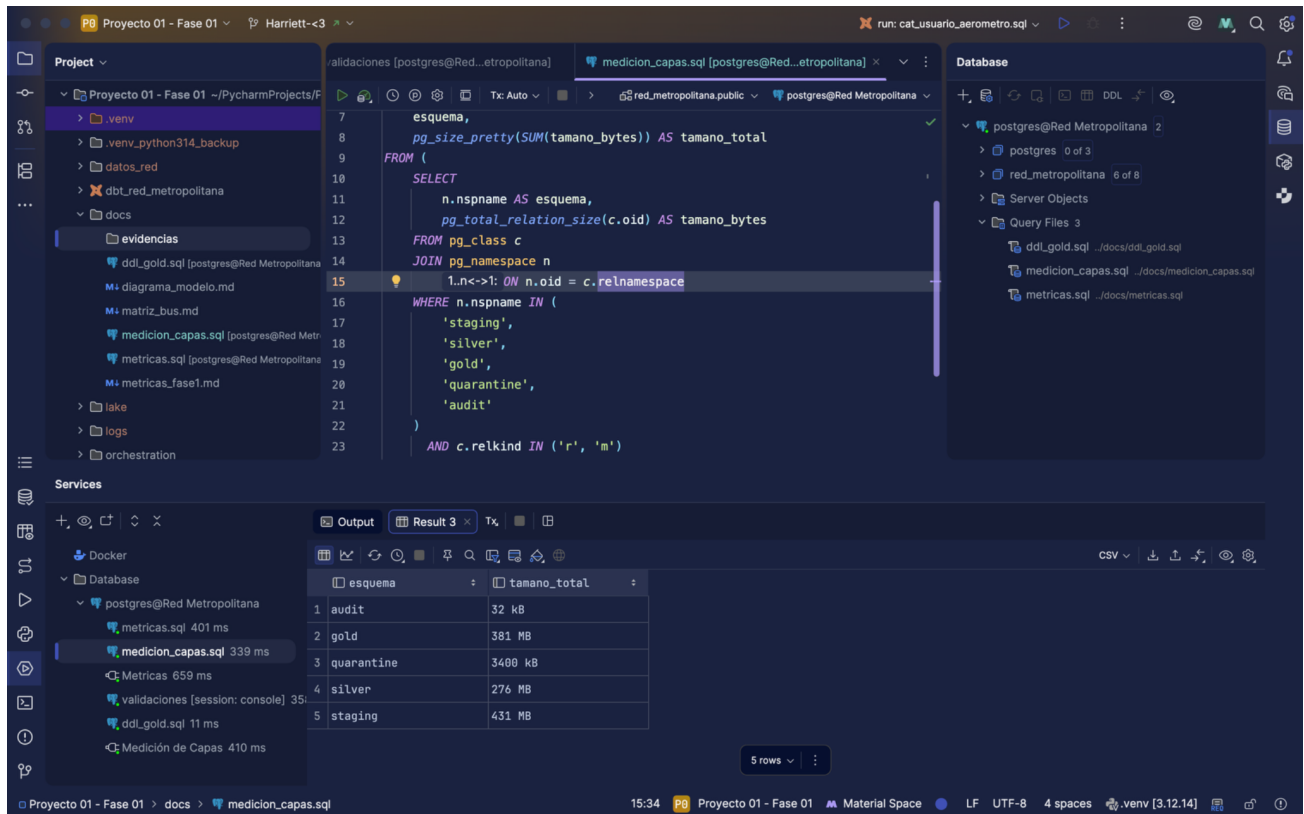
The terminal at the bottom shows the output of the dbt command, indicating that 79 tests passed successfully. The output is as follows:

```
01:31:50 75 of 79 PASS unique_dim_zona_zona_sk ..... [PASS in 0.02s]
01:31:50 79 of 79 START test unique_transmetro_validaciones_validacion_id ..... [RUN]
01:31:50 78 of 79 PASS unique_metrerie_l_viajes_viaje_id ..... [PASS in 0.11s]
01:31:50 77 of 79 PASS unique_fct_viajes_metrerie_l_viaje_sk ..... [PASS in 0.14s]
01:31:50 79 of 79 PASS unique_transmetro_validaciones_validacion_id ..... [PASS in 0.12s]
01:31:50 76 of 79 PASS unique_fct_abordajes_abordaje_sk ..... [PASS in 0.47s]
01:31:50
01:31:50 Finished running 79 data tests in 0 hours 0 minutes and 1.50 seconds (1.50s).
01:31:50
01:31:50 Completed successfully
01:31:50
01:31:50 Done. PASS=79 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=79
```

Anexo 13. Resultado PASS=79, WARN=0, ERROR=0 y SKIP=0.

Tamaño de las capas almacenadas en PostgreSQL

La medición inicial calcula el espacio ocupado por tablas y vistas materializadas dentro de PostgreSQL. Los resultados muestran 431 MB en Staging, 276 MB en Silver, 381 MB en Gold, 3,400 kB en cuarentena y 32 kB en Audit. Bronze no aparece aquí porque se almacena en el sistema de archivos.



The screenshot shows the PyCharm IDE interface with a PostgreSQL query editor and a results pane. The query calculates the size of tables and materialized views across different schemas. The results pane displays the following data:

esquema	tamano_total
audit	32 kB
gold	381 MB
quarantine	3400 kB
silver	276 MB
staging	431 MB

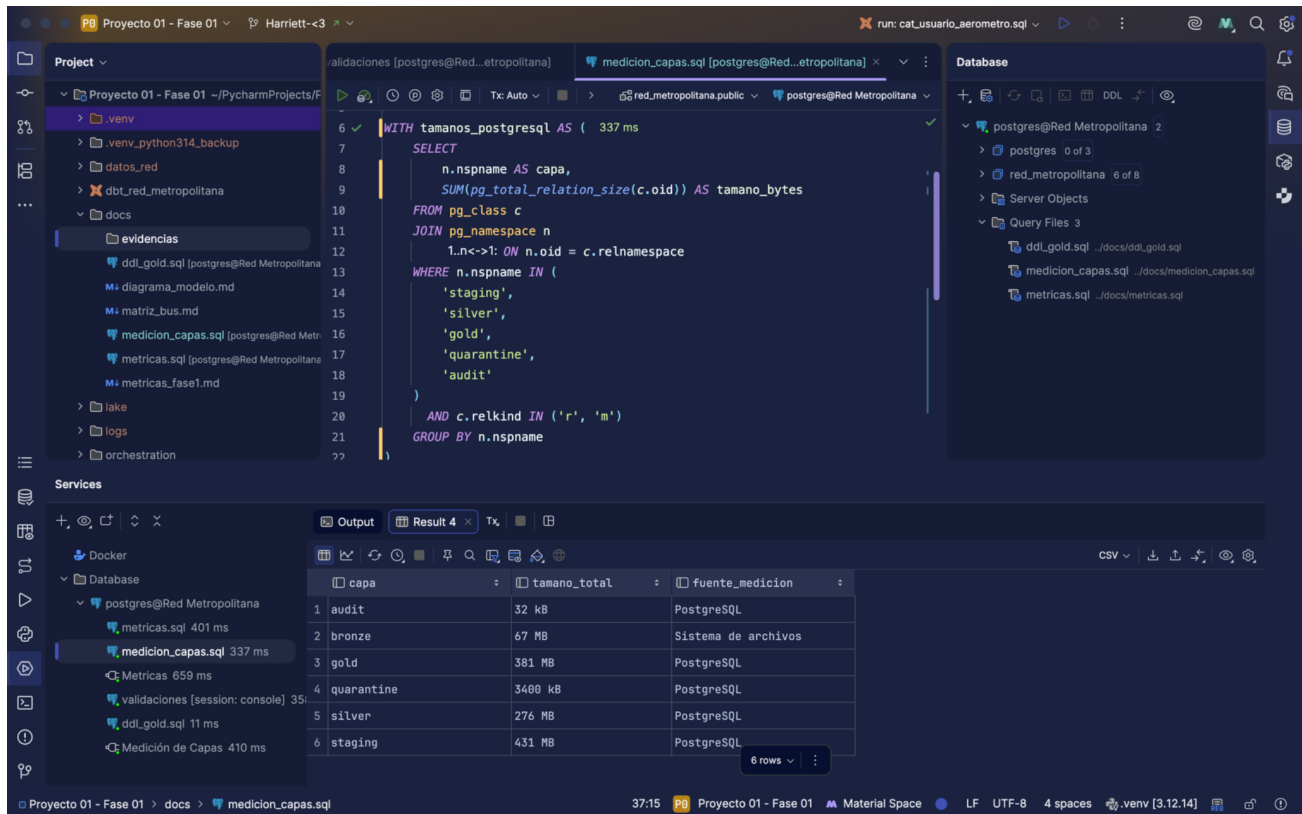
The query editor shows the following SQL code:

```
7 esquema,
8 pg_size_pretty(SUM(tamano_bytes)) AS tamano_total
9 FROM (
10     SELECT
11         n.nspname AS esquema,
12         pg_total_relation_size(c.oid) AS tamano_bytes
13     FROM pg_class c
14     JOIN pg_namespace n
15         ON 1..n->1: ON n.oid = c.relnamespace
16     WHERE n.nspname IN (
17         'staging',
18         'silver',
19         'gold',
20         'quarantine',
21         'audit'
22     )
23     AND c.relkind IN ('r', 'm')
```

Anexo 14. Medición de los esquemas persistidos en PostgreSQL.

Medición consolidada de todas las capas

La consulta final integra la medición de PostgreSQL con el valor obtenido para Bronze. La columna de fuente distingue el origen de cada dato: sistema de archivos para Bronze y PostgreSQL para las demás capas. De esta manera, el reporte presenta en una sola tabla el tamaño de las seis áreas de almacenamiento del pipeline.



The screenshot shows a PyCharm IDE with a PostgreSQL query editor. The query is as follows:

```
WITH tamanos_postgresql AS (
    SELECT
        n.nspname AS capa,
        SUM(pg_total_relation_size(c.oid)) AS tamaño_bytes
    FROM pg_class c
    JOIN pg_namespace n
    ON n.oid = c.relnamespace
    WHERE n.nspname IN (
        'staging',
        'silver',
        'gold',
        'quarantine',
        'audit'
    )
    AND c.relkind IN ('r', 'm')
)
GROUP BY n.nspname
```

The results are displayed in a table with 6 rows:

capa	tamaño_total	fuentes_medicion
1 audit	32 kB	PostgreSQL
2 bronze	67 MB	Sistema de archivos
3 gold	381 MB	PostgreSQL
4 quarantine	3400 kB	PostgreSQL
5 silver	276 MB	PostgreSQL
6 staging	431 MB	PostgreSQL

Anexo 15. Resumen consolidado del tamaño y la fuente de medición por capa.